

InterviewGen AI

Interview Test Report

Candidate Information

Name : Yaswanth Krishna
Email : vykrishna2006@gmail.com
Date : 5/7/2026
Status : Completed

Overall Score

1%

1 / 70 questions correct

Section-wise Results

C/C++	0/20 (0%)
Java	0/20 (0%)
Python	2/20 (10%)
SQL	0/20 (0%)
AWS	0/20 (0%)
EC2	0/20 (0%)
HR	0/20 (0%)

Detailed Question Analysis

Q1. [C/C++] ' INCORRECT – 0/5

Question:

What is a fundamental difference between a C++ pointer and a C++ reference?

Your Answer:

Not answered

Ideal Answer:

Pointers can be null, while references must always refer to a valid object and cannot be null. Additionally, a pointer can be re-assigned to point to a different object after initialization, whereas a reference, once initialized, will always refer to the same object.

AI Feedback:

No answer provided.

Q2. [C/C++] ' INCORRECT – 0/5

Question:

In C++, what is the primary purpose of declaring a member function as `virtual`?

Your Answer:

Not answered

Ideal Answer:

Declaring a member function as `virtual` enables runtime polymorphism. It ensures that the correct overridden version of the function is called based on the actual type of the object being pointed to or referenced, rather than the static type of the pointer or reference. This is typically achieved using a vtable.

AI Feedback:

No answer provided.

Q3. [C/C++] ' INCORRECT – 0/5

Question:

What is a key advantage of using `new` and `delete` operators over `malloc` and `free` functions in C++?

Your Answer:

Not answered

Ideal Answer:

The `new` and `delete` operators automatically invoke constructors and destructors, respectively, for user-defined types, ensuring proper initialization and cleanup. They are also type-safe, returning pointers of the correct type, and can be overloaded by classes to customize memory allocation behavior.

AI Feedback:

No answer provided.

Q4. [C/C++] ' INCORRECT – 0/5

Question:

Consider the declaration `const int\* p;`. What does the `const` keyword modify in this specific context?

Your Answer:

Not answered

Ideal Answer:

In `const int\* p;`, the `const` keyword modifies the data being pointed to. This means that `p` is a pointer to an integer that cannot be modified through `p`. The pointer `p` itself can be changed to point to a different integer.

AI Feedback:

No answer provided.

Q5. [C/C++] ' INCORRECT – 0/5

Question:

What is the core principle behind the C++ idiom 'Resource Acquisition Is Initialization' (RAII)?

Your Answer:

Not answered

Ideal Answer:

RAII leverages object lifetimes to manage resources automatically. Resources are acquired in a constructor and released in the corresponding destructor. This ensures that resources are properly released when the object goes out of scope, regardless of how the scope is exited (e.g., normal return, exception), guaranteeing exception safety and preventing resource leaks.

AI Feedback:

No answer provided.

Q6. [C/C++] ' INCORRECT – 0/5

Question:

What is the primary use case for `#ifndef`/`#define`/`#endif` directives in C/C++ header files?

Your Answer:

Not answered

Ideal Answer:

These directives are primarily used to create 'include guards' or 'header guards.' Their purpose is to prevent the multiple inclusion of the same header file within a single compilation unit, which can lead to redefinition errors for classes, functions, or variables.

AI Feedback:

No answer provided.

Q7. [C/C++] ' INCORRECT – 0/5

Question:

In C++, what is a significant advantage of using templates over `void*` pointers for generic programming?

Your Answer:

Not answered

Ideal Answer:

Templates provide type safety at compile time. They allow writing generic code that works with different types while still enforcing type checks, thus preventing runtime errors that can arise from incorrect type casting with `void*`. Templates also generate specialized code for each type, often leading to better performance.

AI Feedback:

No answer provided.

Q8. [C/C++] ' INCORRECT – 0/5

Question:

What is the primary difference in ownership semantics between `std::unique_ptr` and `std::shared_ptr`?

Your Answer:

Not answered

Ideal Answer:

`std::unique_ptr` implements exclusive ownership, meaning only one `unique_ptr` can own a raw pointer at a time. When the `unique_ptr` goes out of scope, the owned resource is automatically deleted. In contrast, `std::shared_ptr` implements shared ownership, where multiple `shared_ptr` instances can own the same resource, and the resource is deleted only when the last `shared_ptr` owning it goes out of scope or is reset.

AI Feedback:

No answer provided.

Q9. [C/C++] ' INCORRECT – 0/5

Question:

What constitutes a 'data race' in a multithreaded C++ program, according to the C++ standard?

Your Answer:

Not answered

Ideal Answer:

A data race occurs when two or more threads concurrently access the same memory location, at least one of the accesses is a write, and there is no synchronization mechanism (like mutexes or atomic operations) to order these accesses. Data races lead to undefined behavior, making the program's outcome unpredictable.

AI Feedback:

No answer provided.

Q10. [C/C++] ' INCORRECT – 0/5

Question:

How does C++ typically implement dynamic dispatch for virtual functions under the hood?

Your Answer:

Not answered

Ideal Answer:

Dynamic dispatch is commonly implemented using a 'virtual table' (vtable). Each class with virtual functions has a vtable, which is an array of function pointers. Objects of such classes typically contain a hidden pointer (vptr) to their class's vtable. When a virtual function is called via a base class pointer or reference, the vptr is dereferenced to find the appropriate function address in the vtable, allowing the correct derived class function to be executed at runtime.

AI Feedback:

No answer provided.

Q11. [Java] ' INCORRECT – 0/5

Question:

What is the primary role of the Java Virtual Machine (JVM)?

Your Answer:

Not answered

Ideal Answer:

The JVM is an abstract machine that provides a runtime environment in which Java bytecode can be executed. It translates Java bytecode into machine-specific instructions and manages memory (garbage collection) and security for Java applications. It ensures Java's "write once, run anywhere" capability.

AI Feedback:

No answer provided.

Q12. [Java] ' INCORRECT – 0/5

Question:

What is a key difference between an abstract class and an interface in Java?

Your Answer:

Not answered

Ideal Answer:

An abstract class can have both abstract and non-abstract methods, and it can declare instance variables, constructors, and static methods. A class can only extend one abstract class. An interface, on the other hand, only contains abstract methods (before Java 8) or default/static methods (since Java 8) and cannot have instance variables (only constants) or constructors. A class can implement multiple interfaces.

AI Feedback:

No answer provided.

Q13. [Java] ' INCORRECT – 0/5

Question:

What is the main purpose of Java's Garbage Collector?

Your Answer:

Not answered

Ideal Answer:

The Java Garbage Collector automatically manages memory by identifying and reclaiming memory occupied by objects that are no longer referenced by the application. This process frees developers from manual memory deallocation, reducing memory leaks and improving application stability. It primarily targets the heap memory.

AI Feedback:

No answer provided.

Q14. [Java] ' INCORRECT – 0/5

Question:

What is the primary benefit of using Generics in Java?

Your Answer:

Not answered

Ideal Answer:

Generics provide type safety at compile-time by allowing types (classes and interfaces) to be parameters when defining classes, interfaces, and methods. This helps to prevent `ClassCastException` errors at runtime by catching invalid type assignments during compilation. It also eliminates the need for explicit type casting and improves code readability.

AI Feedback:

No answer provided.

Q15. [Java] ' INCORRECT – 0/5

Question:

In Java, what is the fundamental difference between a checked exception and an unchecked exception?

Your Answer:

Not answered

Ideal Answer:

Checked exceptions are exceptions that must be caught or declared in a `throws` clause at compile-time; the compiler enforces their handling. They typically represent predictable problems that a well-written application should anticipate and recover from, like `IOException`. Unchecked exceptions, on the other hand, are runtime exceptions that do not need to be explicitly caught or declared. They often indicate programming errors, such as `NullPointerException` or `ArrayIndexOutOfBoundsException`.

AI Feedback:

No answer provided.

Q16. [Java] ' INCORRECT – 0/5

Question:

What is the `volatile` keyword used for in Java in the context of multithreading?

Your Answer:

Not answered

Ideal Answer:

The `volatile` keyword ensures that a variable's value is always read from main memory and not from a CPU cache, and that any write to a volatile variable is written back to main memory immediately. This guarantees visibility of changes to other threads, preventing stale values. It does not provide atomicity, only visibility.

AI Feedback:

No answer provided.

Q17. [Java] ' INCORRECT – 0/5

Question:

In Java's Stream API, what is the key characteristic that distinguishes an intermediate operation from a terminal operation?

Your Answer:

Not answered

Ideal Answer:

Intermediate operations (like `filter`, `map`, `sorted`) are lazy and return another `Stream`. They don't perform any actual computation until a terminal operation is invoked, allowing for optimization techniques like short-circuiting. Terminal operations (like `forEach`, `collect`, `reduce`, `count`) are eager and produce a non-stream result (e.g., a collection, a value, or `void`), effectively consuming the stream and marking the end of the stream pipeline.

AI Feedback:

No answer provided.

Q18. [Java] ' INCORRECT – 0/5

Question:

Which of the following is a primary benefit of creating immutable objects in Java?

Your Answer:

Not answered

Ideal Answer:

Immutable objects, once created, cannot be modified. This inherent characteristic makes them inherently thread-safe as their state never changes, eliminating the need for synchronization. They are also easier to reason about, can be safely shared without defensive copying, and are good candidates for keys in `HashMap`'s or elements in `HashSet`'s.

AI Feedback:

No answer provided.

Q19. [Java] ' INCORRECT – 0/5

Question:

What is the primary advantage of `ConcurrentHashMap` over `HashMap` in a multithreaded environment?

Your Answer:

Not answered

Ideal Answer:

`ConcurrentHashMap` provides thread-safe operations without synchronizing the entire map, unlike `Collections.synchronizedMap()` or manually synchronizing `HashMap`, which can lead to performance bottlenecks. It achieves this by segmenting the map and locking only parts of it during updates, allowing concurrent reads and concurrent writes to different segments. This offers higher concurrency and better performance under heavy contention.

AI Feedback:

No answer provided.

Q20. [Java] ' INCORRECT – 0/5

Question:

What is the main purpose of lambda expressions in Java 8 and beyond?

Your Answer:

Not answered

Ideal Answer:

Lambda expressions provide a concise way to represent an anonymous function, allowing functional

interfaces to be instantiated more easily. Their main purpose is to enable functional programming in Java, making code more readable, succinct, and easier to maintain, especially when dealing with collections and the Stream API. They eliminate the need for verbose anonymous inner classes for single-method interfaces.

AI Feedback:

No answer provided.

Q21. [Python] ' CORRECT – 2/5

Question:

What will be the output of the following Python code?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

Your Answer:

[1, 2, 3, 4]

Ideal Answer:

Lists are mutable objects in Python. When `b = a` is executed, both `a` and `b` refer to the same list object in memory. Therefore, modifying the list through `b` also affects the list referenced by `a`.

AI Feedback:

The candidate correctly identified the output of the code. This demonstrates a clear understanding of Python's object model, specifically how lists (mutable objects) and variable assignments work, where `b = a` creates a new reference to the *same* list object in memory.

Q22. [Python] ' INCORRECT – 0/5

Question:

Which of the following list comprehensions correctly generates a list of squares for even numbers between 0 and 9 (inclusive)?

Your Answer:

Not answered

Ideal Answer:

The list comprehension `[x**2 for x in range(10) if x % 2 == 0]` iterates through numbers from 0 to 9. The `if x % 2 == 0` condition filters for even numbers, and `x**2` calculates their squares.

AI Feedback:

No answer provided.

Q23. [Python] ' INCORRECT – 0/5

Question:

What will be the output of the following Python code?

```
def func(a, *args, **kwargs):
    print(a)
    print(args)
    print(kwargs)
```

```
func(1, 2, 3, x=4, y=5)
```

Your Answer:

Not answered

Ideal Answer:

`a` receives the first positional argument `1`. `*args` collects all remaining positional arguments into a tuple `(2, 3)`. `**kwargs` collects all keyword arguments into a dictionary `{'x': 4, 'y': 5}`.

AI Feedback:

No answer provided.

Q24. [Python] ' INCORRECT – 0/5

Question:

What is the output of `my_dict.get('c', 0)` if `my_dict = {'a': 1, 'b': 2}`?

Your Answer:

Not answered

Ideal Answer:

The `get()` method returns the value for the specified key if the key is in the dictionary. If the key is not found, it returns the default value provided (in this case, `0`), instead of raising a `KeyError`.

AI Feedback:

No answer provided.

Q25. [Python] ' INCORRECT – 0/5

Question:

Which statement best describes the primary advantage of using a generator expression `(x*x for x in range(10**6))` over a list comprehension `[x*x for x in range(10**6)]` for large datasets?

Your Answer:

Not answered

Ideal Answer:

Generators produce items one at a time and on demand, meaning they don't store the entire sequence in memory. This makes them significantly more memory-efficient than list comprehensions for large datasets, which construct the entire list upfront.

AI Feedback:

No answer provided.

Q26. [Python] ' INCORRECT – 0/5

Question:

Consider the following Python decorator:

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print('Before function call')
        result = func(*args, **kwargs)
        print('After function call')
        return result
    return wrapper
```

```
@my_decorator
def say_hello(name):
    return f'Hello, {name}!'
```

What will be the output when `say_hello('Alice')` is called?

Your Answer:

Not answered

Ideal Answer:

The `@my_decorator` syntax applies `my_decorator` to `say_hello`. When `say_hello('Alice')` is called, it actually invokes the `wrapper` function returned by `my_decorator`. The wrapper prints 'Before function call', then calls the original `say_hello`, prints 'After function call', and finally returns the result of `say_hello`.

AI Feedback:

No answer provided.

Q27. [Python] ' INCORRECT – 0/5

Question:

Given the following class hierarchy, what is the MRO for class D?

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
```

Your Answer:

Not answered

Ideal Answer:

Python's MRO for multiple inheritance follows the C3 linearization algorithm. For D(B, C), it first checks D, then B, then C. The algorithm ensures that base classes are checked before derived classes and that a class appears only once in the MRO. The result is `D -> B -> C -> A -> object`.

AI Feedback:

No answer provided.

Q28. [Python] ' INCORRECT – 0/5

Question:

Which of the following statements about Python's Global Interpreter Lock (GIL) is true?

Your Answer:

Not answered

Ideal Answer:

The GIL ensures that only one native thread can execute Python bytecodes at a time, even on multi-core processors. This simplifies memory management and prevents race conditions on Python objects, but it limits true parallelism for CPU-bound Python code.

AI Feedback:

No answer provided.

Q29. [Python] ' INCORRECT – 0/5

Question:

What is the primary benefit of using a `with` statement with context managers in Python?

Your Answer:

Not answered

Ideal Answer:

The `with` statement ensures that resources are properly acquired and released, even if errors occur. It handles the setup (via `\_\_enter\_\_`) and teardown (via `\_\_exit\_\_`) operations automatically, making code cleaner and more robust for resource management like file I/O or locking.

AI Feedback:

No answer provided.

Q30. [Python] ' INCORRECT – 0/5

Question:

What will be the output of the following Python code?

```
def create_multipliers():
    return [lambda x : i * x for i in range(3)]

for multiplier in create_multipliers():
    print(multiplier(2))
```

Your Answer:

Not answered

Ideal Answer:

This demonstrates Python's late binding for closures. The `i` in the lambda function is not evaluated until the lambda is called. By that time, the loop has completed, and `i` has its final value, which is `2`. So, all lambdas will use `i=2`, resulting in `2 * 2 = 4` for each call.

AI Feedback:

No answer provided.

Q31. [SQL] ' INCORRECT – 0/5

Question:

Which clause is used to filter individual rows based on a specified condition before any grouping or aggregation occurs?

Your Answer:

Not answered

Ideal Answer:

The WHERE clause is used to filter individual rows based on a specified condition before any grouping or aggregation occurs. It applies conditions to each row in the result set, allowing only those that satisfy the condition to be included.

AI Feedback:

No answer provided.

Q32. [SQL] ' INCORRECT – 0/5

Question:

What is the primary difference between an `INNER JOIN` and a `LEFT JOIN`?

Your Answer:

Not answered

Ideal Answer:

An `INNER JOIN` returns only the rows where there is a match in *both* tables based on the join condition. A `LEFT JOIN` (or `LEFT OUTER JOIN`) returns all rows from the left table, and the matching rows from the right table. If there's no match in the right table, `NULL` values are returned for the right table's columns.

AI Feedback:

No answer provided.

Q33. [SQL] ' INCORRECT – 0/5

Question:

When should you use the `HAVING` clause instead of the `WHERE` clause?

Your Answer:

Not answered

Ideal Answer:

The `HAVING` clause is used to filter groups of rows based on a specified condition, typically after aggregation functions have been applied. In contrast, the `WHERE` clause filters individual rows *before* they are grouped. `HAVING` operates on the results of `GROUP BY`, allowing conditions on aggregate values like `COUNT()`, `SUM()`, or `AVG()`.

AI Feedback:

No answer provided.

Q34. [SQL] ' INCORRECT – 0/5

Question:

What is a Common Table Expression (CTE) and why might you use it?

Your Answer:

Not answered

Ideal Answer:

A CTE is a named temporary result set that you can reference within a single `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement. It's defined using the `WITH` clause. CTEs improve readability and maintainability for complex queries, especially when dealing with recursive queries or when you need to break down a complicated query into logical, reusable steps, making the query structure clearer than nested subqueries.

AI Feedback:

No answer provided.

Q35. [SQL] ' INCORRECT – 0/5

Question:

Explain the purpose of the `ROW\_NUMBER()` window function.

Your Answer:

Not answered

Ideal Answer:

`ROW\_NUMBER()` assigns a unique, sequential integer to each row within a partition of a result set, starting from 1 for the first row in each partition. The ordering of rows within the partition is determined by the `ORDER BY` clause within the `OVER()` clause. It's often used to find the 'nth' row, or to de-duplicate data by selecting the first entry based on some criteria.

AI Feedback:

No answer provided.

Q36. [SQL] ' INCORRECT – 0/5

Question:

What is the purpose of a `PRIMARY KEY` constraint?

Your Answer:

Not answered

Ideal Answer:

A `PRIMARY KEY` constraint uniquely identifies each record in a table and ensures data integrity. It must contain unique values for each row and cannot contain `NULL` values. A table can have only one primary key, which can consist of one or more columns.

AI Feedback:

No answer provided.

Q37. [SQL] ' INCORRECT – 0/5

Question:

How do indexes improve query performance, and what are their potential drawbacks?

Your Answer:

Not answered

Ideal Answer:

Indexes improve query performance by providing a quick lookup mechanism for database rows, similar to a book's index. They speed up data retrieval operations (SELECT statements) by reducing the amount of data the database system needs to scan. However, indexes consume disk space and can slow down data modification operations (INSERT, UPDATE, DELETE) because the index itself must also be updated.

AI Feedback:

No answer provided.

Q38. [SQL] ' INCORRECT – 0/5

Question:

Briefly explain the 'Atomicity' property in the context of database transactions (ACID properties).

Your Answer:

Not answered

Ideal Answer:

Atomicity, in the context of ACID properties, means that a transaction is treated as a single, indivisible unit of work. Either all of its operations are completed successfully, or none of them are. If any part of the transaction fails, the entire transaction is rolled back, leaving the database in its state before the transaction began. This ensures that the database state remains consistent and correct.

AI Feedback:

No answer provided.

Q39. [SQL] ' INCORRECT – 0/5

Question:

Which SQL function is generally used to extract only the year from a datetime column named `order\_date`?

Your Answer:

Not answered

Ideal Answer:

The `EXTRACT()` function is a standard SQL function used to retrieve specific parts of a date or time value, such as the year, month, or day. For example, `EXTRACT(YEAR FROM order\_date)` will return the year as an integer from the `order\_date` column. Other databases might use `YEAR()` or `DATE\_PART()` functions.

AI Feedback:

No answer provided.

Q40. [SQL] ' INCORRECT – 0/5

Question:

How do aggregate functions like `COUNT()`, `SUM()`, and `AVG()` generally handle `NULL` values?

Your Answer:

Not answered

Ideal Answer:

By default, most aggregate functions in SQL (like `SUM()`, `AVG()`, `COUNT(column\_name)`, `MIN()`, `MAX()`) ignore `NULL` values in their calculations. For `COUNT(\*)`, it counts all rows regardless of `NULL`s. If you want to include `NULL`s in a count, you can use `COUNT(\*)` or `COUNT(1)`, but specific column counts will omit `NULL`s for that column. `NULL` values are simply excluded from the set of values being aggregated.

AI Feedback:

No answer provided.

Q41. [AWS] ' INCORRECT – 0/5

Question:

Explain the key components of an AWS VPC and how they enable network isolation and connectivity.

Your Answer:

Not answered

Ideal Answer:

A VPC (Virtual Private Cloud) is a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define. Key components include:

- * **CIDR Block**: Defines the IP address range for the VPC.
- * **Subnets**: Partitions the VPC's CIDR block into smaller ranges. Can be public (with an Internet Gateway) or private (without direct internet access).
- * **Internet Gateway (IGW)**: Allows communication between instances in public subnets and the internet.
- * **NAT Gateway/Instance**: Allows instances in private subnets to initiate outbound connections to the internet while preventing inbound connections.
- * **Route Tables**: Determine where network traffic from a subnet or gateway is directed.
- * **Security Groups**: Act as virtual firewalls for instances to control inbound/outbound traffic at the instance level.
- * **Network ACLs (NACLs)**: Act as stateless firewalls for subnets to control inbound/outbound traffic at the subnet level.

AI Feedback:

No answer provided.

Q42. [AWS] ' INCORRECT – 0/5

Question:

Describe different S3 storage classes and when you would choose one over another.

Your Answer:

Not answered

Ideal Answer:

Amazon S3 offers various storage classes optimized for different access patterns and cost requirements:

- * **S3 Standard**: Default, highly durable, available, and performant storage for frequently accessed data.

Good for general-purpose storage, dynamic websites, content distribution.

- * **S3 Intelligent-Tiering**: Automatically moves data between two access tiers (frequent and infrequent) based on access patterns, optimizing costs without performance impact. Ideal for data with unknown or changing access patterns.

- * **S3 Standard-IA (Infrequent Access)**: For long-lived, less frequently accessed data that requires rapid access when needed. Lower storage cost than Standard but higher retrieval cost. Good for backups, disaster recovery, analytics data.

- * **S3 One Zone-IA**: Similar to Standard-IA but stores data in a single Availability Zone, making it cheaper but less resilient to AZ loss. Suitable for reproducible data or secondary backups.

- * **S3 Glacier Instant Retrieval**: For long-lived data that is rarely accessed but requires millisecond retrieval when needed. Cheaper than IA classes, higher retrieval cost.

- * **S3 Glacier Flexible Retrieval**: For archiving data that can be retrieved in minutes to hours. Very low cost. Suitable for long-term archives.

- * **S3 Glacier Deep Archive**: Lowest-cost storage for long-term archives, with retrieval times of hours. Compliance archives, highly cost-sensitive long-term backups.

AI Feedback:

No answer provided.

Q43. [AWS] ' INCORRECT – 0/5

Question:

How would you design a highly available and scalable web application on AWS using EC2?

Your Answer:

Not answered

Ideal Answer:

To design a highly available and scalable web application on AWS using EC2, I would incorporate the following:

- * **Multi-AZ Deployment**: Deploy EC2 instances across multiple Availability Zones (AZs) to protect against single AZ failures.

- * **Auto Scaling Groups (ASG)**: Use ASGs to automatically adjust the number of EC2 instances based on demand (e.g., CPU utilization, network I/O, custom metrics). This ensures scalability and helps maintain availability by replacing unhealthy instances.

- * **Elastic Load Balancer (ELB)**: Place an ELB (e.g., Application Load Balancer for HTTP/S traffic) in front of the ASG to distribute incoming traffic across healthy instances in different AZs. The ELB also handles SSL termination and health checks.

- * **Stateless Application Design**: Ensure application servers are stateless, pushing session management to external services like ElastiCache (Redis) or DynamoDB. This allows instances to be added or removed easily without data loss.

- * **Managed Database Services**: Use Amazon RDS (Multi-AZ deployment for HA) or DynamoDB for database needs, offloading database management and ensuring data availability.

- * **Centralized Logging & Monitoring**: Use CloudWatch, CloudTrail, and perhaps a service like Kinesis Firehose + S3/Elasticsearch for centralized logging and monitoring to quickly identify and troubleshoot issues.

AI Feedback:

No answer provided.

Q44. [AWS] ' INCORRECT – 0/5

Question:

When would you choose Amazon RDS over Amazon DynamoDB, and vice-versa? Provide examples.

Your Answer:

Not answered

Ideal Answer:

- * **Choose Amazon RDS (Relational Database Service) when:**
 - * You need a traditional relational database (SQL) with ACID compliance.
 - * Your data has a well-defined, rigid schema and complex joins are frequent.
 - * You need strong data consistency and transaction support.
 - * Examples: E-commerce platforms with complex product catalogs and order processing, financial applications requiring strict data integrity, content management systems.
- * **Choose Amazon DynamoDB (NoSQL Database) when:**
 - * You need a highly scalable, low-latency, non-relational database.
 - * Your data model is flexible and schema-less, or semi-structured.
 - * You require massive scale for read/write operations (millions of requests per second) and predictable performance.
 - * You need automatic scaling, high availability, and built-in replication across multiple AZs.
 - * Examples: Gaming leaderboards, IoT device data streams, real-time bidding platforms, user profiles for large-scale applications.

AI Feedback:

No answer provided.

Q45. [AWS] ' INCORRECT – 0/5

Question:

Explain the benefits and use cases of AWS Lambda. What are some limitations?

Your Answer:

Not answered

Ideal Answer:

- * **Benefits of AWS Lambda:**
 - * **No Server Management:** You don't provision or manage servers; AWS handles all infrastructure.
 - * **Automatic Scaling:** Scales automatically with demand, from a few requests per day to thousands per second.
 - * **Cost-Effective:** Pay only for the compute time consumed, charged per 100ms. No idle costs.
 - * **High Availability:** Built-in fault tolerance and availability.
 - * **Event-Driven:** Easily triggered by events from over 200 AWS services (S3, DynamoDB, API Gateway, Kinesis, etc.).
- * **Use Cases:**
 - * **Data Processing:** Thumbnail generation on S3 uploads, ETL processes.
 - * **Web Backends:** RESTful APIs via API Gateway, handling user requests.
 - * **Real-time Stream Processing:** Processing data from Kinesis or DynamoDB Streams.
 - * **Scheduled Tasks:** Running cron jobs.
 - * **Chatbots/Voice Assistants:** Processing requests from Alexa, Slack.
- * **Limitations:**
 - * **Execution Duration:** Default maximum execution time is 15 minutes.
 - * **Memory/Disk Limits:** Restricted memory (up to 10GB) and ephemeral disk space (/tmp up to 10GB).
 - * **Cold Starts:** Initial invocations of infrequently used functions might experience a slight delay due to container initialization.
 - * **Concurrency Limits:** Default regional concurrency limits can be hit if not managed.
 - * **Debugging Complexity:** Debugging distributed serverless architectures can be more challenging.

AI Feedback:

No answer provided.

Q46. [AWS] ' INCORRECT – 0/5

Question:

What is AWS IAM, and how does it help secure your AWS resources?

Your Answer:

Not answered

Ideal Answer:

AWS Identity and Access Management (IAM) is a web service that helps you securely control access to AWS resources. It allows you to manage users, groups, roles, and their permissions.

* **Key Concepts**:

- * **Users**: Entities representing a person or application that interacts with AWS.

- * **Groups**: Collections of IAM users, simplifying permission management for multiple users.

- * **Roles**: IAM entities that define a set of permissions for making AWS service requests. They are typically assumed by AWS services (e.g., EC2 instance role) or trusted external identities (e.g., cross-account access).

- * **Policies**: Documents that define permissions (what actions are allowed or denied on which resources). Policies can be attached to users, groups, or roles.

* **How it helps secure resources**:

- * **Principle of Least Privilege**: IAM enables you to grant only the necessary permissions required to perform a task, minimizing potential security risks.

- * **Granular Access Control**: Policies allow very specific control over which resources can be accessed and what actions can be performed (e.g., "Allow S3:GetObject on bucket 'my-app-data' only").

- * **Identity Federation**: Allows existing enterprise identities (e.g., Active Directory) to access AWS resources without recreating users in IAM.

- * **MFA (Multi-Factor Authentication)**: Can be enforced for IAM users to add an extra layer of security.

- * **Auditability**: Integrates with AWS CloudTrail to log all API calls, including who made them and from where, aiding in security audits.

AI Feedback:

No answer provided.

Q47. [AWS] ' INCORRECT – 0/5

Question:

Differentiate between AWS CloudWatch and AWS CloudTrail. When would you use each?

Your Answer:

Not answered

Ideal Answer:

* **AWS CloudWatch**:

- * **What it is**: A monitoring and observability service that provides data and actionable insights to monitor your applications, respond to system-wide performance changes, and optimize resource utilization.

- * **What it does**: Collects and tracks metrics (e.g., CPU utilization, network I/O, custom application metrics), creates dashboards, sets alarms based on metric thresholds, and reacts to events.

- * **When to use**: For operational monitoring, performance analysis, health checks, capacity planning, and triggering actions based on application or infrastructure performance. E.g., setting an alarm if EC2 CPU utilization exceeds 80% for 5 minutes.

* **AWS CloudTrail**:

- * **What it is**: A service that enables governance, compliance, operational auditing, and risk auditing of your AWS account.

- * **What it does**: Logs API calls made by users, roles, or AWS services in your account. These logs include who made the call, when, from where, and what resources were affected.

- * **When to use**: For security analysis, tracking changes to AWS resources, troubleshooting operational issues (e.g., identifying who deleted a resource), and meeting compliance requirements. E.g., finding out who stopped an EC2 instance or changed an S3 bucket policy.

AI Feedback:

No answer provided.

Q48. [AWS] ' INCORRECT – 0/5

Question:

Explain different disaster recovery strategies on AWS and factors influencing your choice.

Your Answer:

Not answered

Ideal Answer:

AWS offers several disaster recovery (DR) strategies, each with different RTO (Recovery Time Objective) and RPO (Recovery Point Objective) and cost implications:

- * **Backup and Restore**: Simplest and cheapest. Data is backed up to S3 (e.g., using EBS snapshots, RDS snapshots). If disaster strikes, restore from backups to a new environment. High RTO/RPO.

- * **Use case**: Non-critical applications, dev/test environments.

- * **Pilot Light**: Core infrastructure is always running in the DR region (e.g., database, minimal EC2 instances), ready to scale up. Data is continuously replicated. Moderate RTO/RPO and cost.

- * **Use case**: Business-critical applications that can tolerate some downtime.

- * **Warm Standby**: A scaled-down but fully functional version of the application is running in the DR region. Data is continuously replicated. Lower RTO/RPO and higher cost than pilot light.

- * **Use case**: Critical applications requiring minimal downtime.

- * **Multi-Site (Hot/Active-Active)**: Full production environment runs in multiple regions simultaneously, actively serving traffic. Data is replicated in real-time. Lowest RTO/RPO but highest cost.

- * **Use case**: Mission-critical applications where any downtime or data loss is unacceptable.

- * **Factors influencing choice**:

- * **RTO (Recovery Time Objective)**: Maximum acceptable downtime.

- * **RPO (Recovery Point Objective)**: Maximum acceptable data loss.

- * **Cost**: Each strategy has different infrastructure and data transfer costs.

- * **Complexity**: Implementation and management effort.

- * **Compliance Requirements**: Industry regulations might mandate specific RTO/RPO.

AI Feedback:

No answer provided.

Q49. [AWS] ' INCORRECT – 0/5

Question:

What are the main differences between AWS ECS and EKS? When would you choose one over the other?

Your Answer:

Not answered

Ideal Answer:

Both ECS and EKS are container orchestration services on AWS.

- * **AWS ECS (Elastic Container Service)**:

- * **What it is**: A highly scalable, high-performance container orchestration service that supports Docker containers. AWS manages the control plane.

- * **Key Features**: Simpler to set up and operate as it's an AWS-native service. Deep integration with other AWS services. Can use EC2 instances or AWS Fargate (serverless compute for containers).

- * **When to choose**: When you need a fully managed container orchestration solution that is tightly integrated with AWS, prefer a simpler operational model, or are already heavily invested in the AWS ecosystem. Good for teams that want to focus purely on applications, not Kubernetes complexities.

- * **AWS EKS (Elastic Kubernetes Service)**:

- * **What it is**: A managed Kubernetes service that makes it easy to deploy, manage, and scale containerized applications using Kubernetes on AWS. AWS manages the Kubernetes control plane.

- * **Key Features**: Provides the full power and flexibility of open-source Kubernetes. Enables portability of applications across hybrid and multi-cloud environments. Requires more Kubernetes specific knowledge. Can use EC2 instances, Fargate, or Bottlerocket for worker nodes.

- * **When to choose**: When you need Kubernetes' rich feature set, want to leverage the vibrant Kubernetes ecosystem, need multi-cloud or hybrid cloud portability, or your team already has strong Kubernetes expertise. Ideal for organizations standardizing on Kubernetes across different environments.

AI Feedback:

No answer provided.

Q50. [AWS] ' INCORRECT – 0/5

Question:

What are some strategies for optimizing costs in AWS?

Your Answer:

Not answered

Ideal Answer:

Cost optimization on AWS is a continuous process involving several strategies:

- * **Right-sizing Instances**: Regularly analyze resource utilization (CPU, memory, network) using CloudWatch metrics to ensure EC2 instances, RDS databases, etc., are not over-provisioned. Downgrade to smaller instance types when appropriate.
- * **Leveraging Reserved Instances (RIs) or Savings Plans**: For stable, predictable workloads, RIs and Savings Plans offer significant discounts (up to 72%) compared to On-Demand pricing for commitments of 1 or 3 years.
- * **Utilizing Spot Instances**: For fault-tolerant, flexible, and interruptible workloads (e.g., batch processing, data analytics), Spot Instances can provide up to 90% savings compared to On-Demand prices.
- * **Adopting Serverless Architectures**: Services like AWS Lambda, S3, DynamoDB, and Fargate are "pay-per-use" or "pay-per-request," eliminating idle costs and reducing operational overhead.
- * **Optimizing S3 Storage**: Use appropriate S3 storage classes (Intelligent-Tiering, Standard-IA, Glacier) based on data access patterns, and implement lifecycle policies to transition or expire old data.
- * **Eliminating Idle/Unused Resources**: Identify and terminate unused EC2 instances, EBS volumes, old snapshots, unattached Elastic IPs, and idle RDS instances.
- * **Data Transfer Costs**: Minimize data transfer out of AWS regions or across AZs when possible, as egress costs can be significant. Use CloudFront for content delivery.
- * **Monitoring and Budgeting**: Use AWS Cost Explorer, Budgets, and Cost & Usage Reports to track spending, identify trends, and set alerts for budget overruns.
- * **Architectural Efficiency**: Design architectures for efficiency, leveraging services like SQS for decoupling components, which can reduce the need for constantly running compute resources.

AI Feedback:

No answer provided.

Q51. [EC2] ' INCORRECT – 0/5

Question:

Explain what Amazon EC2 is and describe the essential steps to launch a new EC2 instance.

Your Answer:

Not answered

Ideal Answer:

Amazon EC2 (Elastic Compute Cloud) provides resizable compute capacity in the cloud. It allows users to rent virtual servers (instances) on which to run their own computer applications. The essential steps to launch an instance include: choosing an Amazon Machine Image (AMI), selecting an instance type, configuring instance details (VPC, subnet, IAM role), adding storage (EBS volumes), configuring security groups, and reviewing/launching.

AI Feedback:

No answer provided.

Q52. [EC2] ' INCORRECT – 0/5

Question:

Differentiate between various EC2 instance families (e.g., General Purpose, Compute Optimized, Memory Optimized, Storage Optimized, Accelerated Computing) and provide a scenario where you would choose each.

Your Answer:

Not answered

Ideal Answer:

General Purpose (M, T) for balanced workloads like web servers. Compute Optimized (C) for compute-

intensive tasks like HPC or video encoding. Memory Optimized (R, X, U) for large in-memory databases or analytics. Storage Optimized (I, D, H) for high I/O NoSQL databases or data warehousing. Accelerated Computing (P, G, F) for machine learning or graphics rendering using GPUs/FPGAs.

AI Feedback:

No answer provided.

Q53. [EC2] ' INCORRECT – 0/5

Question:

Explain the key differences between EC2 Security Groups and Network Access Control Lists (Network ACLs), and when you would use one over the other or both.

Your Answer:

Not answered

Ideal Answer:

Security Groups are stateful virtual firewalls at the instance level, allowing only 'allow' rules. Network ACLs are stateless virtual firewalls at the subnet level, allowing both 'allow' and 'deny' rules evaluated in order. Security Groups are mandatory for instance security; Network ACLs provide an optional, additional layer of subnet security, often used for more granular control or deny rules.

AI Feedback:

No answer provided.

Q54. [EC2] ' INCORRECT – 0/5

Question:

Describe the differences between Amazon EBS (Elastic Block Store) and EC2 Instance Store volumes. When would you choose one over the other?

Your Answer:

Not answered

Ideal Answer:

EBS volumes are persistent, network-attached storage; data remains even if the instance is stopped or terminated. They offer various performance types and can be snapshotted. Instance Store volumes are ephemeral storage physically attached to the host; data is lost when the instance is stopped, terminated, or fails, but they offer very high I/O performance. Choose EBS for persistent data, and Instance Store for temporary data, caches, or replicated data that can tolerate loss.

AI Feedback:

No answer provided.

Q55. [EC2] ' INCORRECT – 0/5

Question:

Explain how an Auto Scaling Group (ASG) works and list the key components required to configure one for an EC2 application.

Your Answer:

Not answered

Ideal Answer:

An ASG automatically adjusts the number of EC2 instances based on demand or schedules, ensuring desired capacity. Key components include: a Launch Template (or Launch Configuration) defining instance parameters, Min/Max/Desired capacity, VPC and subnets, Scaling Policies (e.g., target tracking, scheduled) often linked to CloudWatch alarms, and Health Checks. Integration with a Load Balancer's Target Group is common.

AI Feedback:

No answer provided.

Q56. [EC2] ' INCORRECT – 0/5

Question:

Compare and contrast On-Demand, Reserved, and Spot EC2 instances, highlighting their primary use cases and cost implications.

Your Answer:

Not answered

Ideal Answer:

On-Demand instances offer pay-as-you-go pricing, suitable for irregular or unpredictable workloads. Reserved Instances (RIs) provide significant discounts for a 1 or 3-year commitment, ideal for stable, continuous workloads. Spot Instances offer deep discounts (up to 90%) by bidding on unused capacity, but can be interrupted; best for fault-tolerant, flexible workloads like batch processing. On-Demand is highest cost, followed by RIs, then Spot is lowest.

AI Feedback:

No answer provided.

Q57. [EC2] ' INCORRECT – 0/5

Question:

How would you monitor the performance and health of your EC2 instances? What metrics are important, and what tools would you use?

Your Answer:

Not answered

Ideal Answer:

I would primarily use Amazon CloudWatch. Important metrics include CPU Utilization, Network In/Out, Disk Read/Write Ops/Bytes (default). For deeper insights like memory or disk space, I'd install the CloudWatch Agent for custom metrics. I would configure CloudWatch Alarms on critical metrics to trigger notifications (SNS) or automated actions, and use CloudWatch Dashboards for visual monitoring. CloudWatch Logs for log aggregation, and AWS Systems Manager for operational insights and management.

AI Feedback:

No answer provided.

Q58. [EC2] ' INCORRECT – 0/5

Question:

An EC2 instance that was previously running fine is now unreachable via SSH/RDP. What steps would you take to diagnose and resolve the issue?

Your Answer:

Not answered

Ideal Answer:

1. Check Instance Status Checks (System and Instance) in EC2 console. 2. Verify Security Group rules allow inbound SSH/RDP from your IP. 3. Check Network ACLs on the subnet allow relevant traffic. 4. Review System Log/Console Output in the EC2 console for boot issues. 5. Monitor CloudWatch metrics (CPU, memory, disk I/O) for resource exhaustion. 6. Use AWS Systems Manager Session Manager (if configured) for in-OS troubleshooting. 7. As a last resort, reboot or stop/start the instance. If data recovery is critical, detach the root EBS volume and attach it to another healthy instance for inspection.

AI Feedback:

No answer provided.

Q59. [EC2] ' INCORRECT – 0/5

Question:

What are EC2 Placement Groups, and when would you use each type (Cluster, Spread, Partition)?

Your Answer:

Not answered

Ideal Answer:

EC2 Placement Groups allow influencing instance placement for specific reliability, performance, or availability needs. Cluster Placement Groups co-locate instances within an AZ for low-latency networking (e.g., HPC). Spread Placement Groups distribute instances across distinct hardware racks within an AZ to reduce correlated failures (e.g., critical small-scale applications). Partition Placement Groups divide instances into logical partitions across distinct racks for large distributed systems (e.g., HDFS, Cassandra) to minimize correlated failures.

AI Feedback:

No answer provided.

Q60. [EC2] ' INCORRECT – 0/5

Question:

You need to deploy a fleet of EC2 instances with a specific OS configuration, pre-installed software, and custom scripts. How would you achieve this efficiently, and what considerations are there for sharing this configuration?

Your Answer:

Not answered

Ideal Answer:

To achieve this efficiently, I would create a 'Golden AMI'. This involves launching a base instance, configuring it with the desired OS settings, installing software, and adding custom scripts. Then, I would create an AMI from this configured instance. For deployment, this custom AMI would be used in Auto Scaling Groups or Launch Templates. Considerations for sharing include: sharing within the same AWS account, explicitly sharing with specific AWS account IDs, and understanding that AMIs are region-specific and must be copied to other regions. Public sharing is generally discouraged for security reasons. Regular updates and rebuilding of AMIs are crucial for security patches.

AI Feedback:

No answer provided.

Q61. [HR] ' INCORRECT – 0/5

Question:

Describe a situation where you had a strong disagreement with a team member or a superior about a technical approach or solution. How did you handle it?

Your Answer:

Not answered

Ideal Answer:

A good answer details the technical disagreement, demonstrates active listening to the other's perspective, articulates their own reasoning with data or evidence, and shows an ability to compromise or align with the team's final decision for the project's success. It emphasizes professional communication and focuses on the best outcome for the project rather than personal victory.

AI Feedback:

No answer provided.

Q62. [HR] ' INCORRECT – 0/5

Question:

Tell me about a significant technical project or feature where you faced a major setback or failure. What happened, how did you react, and what did you learn from it?

Your Answer:

Not answered

Ideal Answer:

The candidate should clearly explain the technical challenge or failure, take responsibility if applicable, describe their problem-solving and debugging process, and outline the corrective actions taken. Crucially, they must articulate specific lessons learned (e.g., improved testing, better communication, different architectural considerations) and how they applied these learnings to subsequent work.

AI Feedback:

No answer provided.

Q63. [HR] ' INCORRECT – 0/5

Question:

Imagine you're working on a critical bug fix for a production system, and simultaneously, your manager assigns you a high-priority new feature request. How would you prioritize your tasks and communicate your plan?

Your Answer:

Not answered

Ideal Answer:

A strong answer involves evaluating the immediate impact and urgency of both tasks, consulting with the manager or relevant stakeholders for clarification on priorities, and proposing a reasoned plan. This might include time-boxing, partial completion, delegation (if possible), or escalating to get more resources, always coupled with clear communication about expected timelines and potential trade-offs.

AI Feedback:

No answer provided.

Q64. [HR] ' INCORRECT – 0/5

Question:

Describe a time you had to quickly learn a new programming language, framework, or technical skill for a project. What was your approach to learning, and how did you apply it?

Your Answer:

Not answered

Ideal Answer:

The candidate should describe the specific technology and the project context. They should detail their proactive learning process (e.g., reading documentation, online courses, personal projects, seeking mentorship), discuss any challenges faced, and demonstrate how they successfully integrated the new skill to contribute effectively to the project.

AI Feedback:

No answer provided.

Q65. [HR] ' INCORRECT – 0/5

Question:

Tell me about a project where the technical requirements were vague or changed frequently. How did you handle this ambiguity and ensure a successful outcome?

Your Answer:

Not answered

Ideal Answer:

A good response highlights proactive communication skills: asking clarifying questions, scheduling regular check-ins with stakeholders, documenting assumptions, and iteratively refining solutions. It demonstrates adaptability, a structured approach to breaking down complex problems, and the ability to manage expectations while navigating evolving requirements.

AI Feedback:

No answer provided.

Q66. [HR] ' INCORRECT – 0/5

Question:

Recount a time when you received constructive criticism on your technical work (e.g., code review, design document). How did you react and what did you do with that feedback?

Your Answer:

Not answered

Ideal Answer:

The candidate should show receptiveness to feedback, even if initially challenging. They should describe how they processed the criticism objectively, asked clarifying questions, and took concrete steps to incorporate the feedback (e.g., refactoring code, revising a design, seeking further learning). It demonstrates humility, a growth mindset, and a commitment to quality.

AI Feedback:

No answer provided.

Q67. [HR] ' INCORRECT – 0/5

Question:

Describe a situation where you took initiative to mentor a less experienced technical colleague or led a small technical initiative within your team. What was the impact?

Your Answer:

Not answered

Ideal Answer:

A strong answer outlines the specific scenario, the actions taken to mentor or lead, challenges encountered, and how they were overcome. The candidate should clearly articulate the positive impact on the individual (e.g., improved skills, confidence), the team (e.g., increased productivity, knowledge sharing), or the project (e.g., successful feature delivery, process improvement).

AI Feedback:

No answer provided.

Q68. [HR] ' INCORRECT – 0/5

Question:

Tell me about a challenging situation where you had to interact with a non-technical stakeholder or client regarding a complex technical issue. How did you manage their expectations and ensure understanding?

Your Answer:

Not answered

Ideal Answer:

The candidate should demonstrate strong communication skills by explaining how they translated complex technical jargon into easily understandable terms for a non-technical audience. They should highlight active listening, empathy, managing expectations regarding timelines or limitations, and collaboratively finding solutions that address both technical feasibility and business needs.

AI Feedback:

No answer provided.

Q69. [HR] ' INCORRECT – 0/5

Question:

Describe a situation where you encountered a potential ethical dilemma or a conflict of interest in a technical project or decision. How did you approach it?

Your Answer:

Not answered

Ideal Answer:

A good response demonstrates a strong ethical compass. The candidate should describe the specific dilemma, how they recognized its ethical implications, and the steps they took to address it (e.g., consulting company policy, speaking with a manager or HR, prioritizing data privacy or security). It highlights integrity and professional responsibility in their decision-making process.

AI Feedback:

No answer provided.

Q70. [HR] ' INCORRECT – 0/5

Question:

Beyond your assigned project tasks, what technical contributions have you made to your team or organization (e.g., improving processes, tooling, documentation, knowledge sharing)?

Your Answer:

Not answered

Ideal Answer:

The candidate should provide concrete examples of initiatives they took outside their core responsibilities. This could include automating tedious tasks, creating better internal documentation, developing a useful tool, or organizing knowledge-sharing sessions. They should clearly articulate the problem solved, the actions taken, and the positive impact on the team's efficiency, quality, or collaboration.

AI Feedback:

No answer provided.
